

rakesh yadav

Project_Report ofPython.docx

 SRM University AP Amravati

Document Details

Submission ID

trn:oid:::8044:92723071

Submission Date

Apr 24, 2025, 6:41 PM GMT+5:30

Download Date

Apr 24, 2025, 6:42 PM GMT+5:30

File Name

Project_Report ofPython.docx

File Size

5.4 MB

16 Pages**2,879 Words****17,759 Characters**

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Small Matches (less than 10 words)

Match Groups

-  **8** Not Cited or Quoted 8%
Matches with neither in-text citation nor quotation marks
-  **0** Missing Quotations 0%
Matches that are still very similar to source material
-  **0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
-  **0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 5%  Internet sources
- 1%  Publications
- 7%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 8** Not Cited or Quoted 8%
Matches with neither in-text citation nor quotation marks
- 0** Missing Quotations 0%
Matches that are still very similar to source material
- 0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
- 0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 5% Internet sources
- 1% Publications
- 7% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	Osmania University, Hyderabad on 2022-02-23	2%
2	Submitted works	srmap on 2022-12-13	1%
3	Submitted works	srmap on 2023-12-07	1%
4	Internet	pdfslide.net	1%
5	Submitted works	srmap on 2023-11-25	<1%
6	Internet	pdfcookie.com	<1%
7	Submitted works	Visvesvaraya Technological University, Belagavi on 2021-02-27	<1%
8	Submitted works	srmap on 2023-12-02	<1%

AI-Based Chatbot

Project submitted to the

SRM University – AP, Andhra Pradesh

for the partial fulfillment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Sciences

Submitted by

Candidate Name

Krishna Khatiwada – AP23110011547

Aayush Kandel – AP23110011679

Rakesh Yadav – AP23110011678

Aniket Jha – AP23110011553



Under the Guidance of

Ms. V. Veda Sri, Ms. Kavitha Rani Karnena

Lecturer, Department of CSE

SRM University-AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

[April, 2025]

Certificate

Date: _____

This is to certify that the work present in this Project entitled “**AI-Based Chatbot**” has been carried out by

[

- **Krishna Khatiwada** (Roll No.: AP23110011547)
- **Rakesh Yadav** (Roll No.: AP23110011678)
- **Aayush Kandel** (Roll No.: AP23110011679)
- **Aniket Jha** (Roll No.: AP23110011553)

]

under my/our supervision. The work is genuine, original, and suitable for submission to the SRM University – AP for the award of Bachelor of Technology/Master of Technology in **School of Engineering and Sciences**.

Supervisor

(Signature)

Prof. / Dr. **Kavitha Rani Karnena**

Designation,

Affiliation.

Acknowledgement

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success.

I am extremely grateful and express my profound gratitude and indebtedness to my project guide, **Ms. Kavitha Rani Karnena**, Department of Computer Science & Engineering, SRM University, Andhra Pradesh, for her kind help and for giving me the necessary guidance and valuable suggestions in completing this project work.

Student Name & Roll.No:

- **Krishna Khatiwada** (Roll No.: AP23110011547)
- **Rakesh Yadav** (Roll No.: AP23110011678)
- **Aayush Kandel** (Roll No.: AP23110011679)
- **Aniket Jha** (Roll No.: AP23110011553)

Abstract

This project report details the development of a chatbot system "CHATBOT" using Flask, Google Gemini API, and MongoDB as a backup to the file system. The motivation behind the project is increasing interest in intelligent assistants that have the capability to process user requests and provide valid answers in real-time. The chatbot leverages state-of-the-art language models using Google's Gemini API to comprehend and respond naturally to user messages. The main strategy was to create a web-based interface with Flask, manage session data and conversation histories in MongoDB, and ensure reliability by falling back to local JSON files for storage. Robustness is ensured through comprehensive logging and organized exception handling. RESTful API endpoints for interaction, history, and health checks are also included in the implementation. The chatbot was tested for responsiveness, accuracy, and system health status. Not only is this project centered around the real-world usage of AI APIs, but it also demonstrates scalable and fault-tolerant backend design. Potential future enhancements involve the inclusion of multilingual conversation support, UI/UX enhancement, and speech-to-text support.

8

1. Table of Contents

Certificate	i
Acknowledgements	iii
Table of Contents	v
Abstract	vii

4

1.	6	
1.1	Error! Bookmark not defined.	
1.1.1	Error! Bookmark not defined.	
2.	Error! Bookmark not defined.	
2.1	Error! Bookmark not defined.	
2.1.1	Error! Bookmark not defined.	
3.	Error! Bookmark not defined.	
4.	Error! Bookmark not defined.	
5.	Discussion and Conclusion	Error! Bookmark not defined.
6.	Future Work	10
	References	11

1. Introduction



The AI Chatbot is a web-based conversation platform that aims to transform human-computer interaction. Built with the Flask web framework and fueled by Google's sophisticated Gemini API, it leverages the power of large language models to provide smart, real-time, and context-sensitive conversations. Beyond mere chatbot capabilities, this assistant has dynamic, human conversation and adjusts its responses to the conversation flow in order to provide a more natural and smoother user experience. The assistant is versatile to be used as an automated customer support aid, as a personal AI tutor, or as an interactive information assistant. Its modular, flexible design promotes ease of use by end-users and painless extensibility for developers, with a tidy front-end UI, organized back-end code, MongoDB support for persistent storage, and a safe file-based fallback mechanism to minimize data loss. Essentially, the AI Chat Assistant is a straightforward and scalable solution for driving user engagement, delivering instant support, and implementing advanced conversational AI across a range of real-world use cases.

1.1 Background

In the quickly changing digital era, artificial intelligence has become an integral part of contemporary communication offerings. With the growing adoption by industries of using AI to streamline processes and enhance user interaction, chatbots are becoming vital agents across sectors ranging from customer care to education, healthcare, and e-commerce. Classic chatbots tend to struggle with narrow awareness of context and rigid responses, though.

Embracing these limitations, the creators set out to develop a more sophisticated conversational platform—one that marries natural language processing (NLP) with contemporary web technologies to provide extremely responsive and contextually aware user interactions. The addition of Google Gemini AI, a cutting-edge large language model, enables the assistant to understand user intent, recall previous interactions, and generate advanced responses akin to human dialogue.

1.2 Significance and Context

The AI Chat Assistant is a milestone towards closing the gap between intelligent automation and users. Its significance is in:

Intelligent Customer Interaction: Supporting users with questions immediately, improving satisfaction, and easing the burden on human support agents.

Contextual Education Assistance: Being an AI tutor capable of remembering continuing academic conversations, explaining, and walking users through learning procedures.

Business Flexibility: Being easily adaptable to particular domains, allowing organizations to adopt and implement it for their own purposes.

In addition, the application provides reliability and data persistence through a hybrid storage mechanism based on MongoDB with file-based fallback, which makes it robust against database failures. This renders the system scalable and fail-safe, making it ideal for real-world production environments.

1.3 Scope and Purpose

The main aim of this project is to develop a conversational AI system that gives correct, quick, and context-aware answers to user questions using a web interface. It has support for multiple sessions of the user, chat history, and can handle multiple domains.

The project scope is:

- **Real-time Chat Interface:** Developing real-time, smooth conversation with users using the web.
- **Persistent Session Management:** Maintaining chat history through Flask-Session, irrespective of browser refresh or reconnection.
- **Smart Backend Architecture:** Using Flask and Gemini API for routing and smart response generation.
- **Scalable Data Storage:** Storing main data using MongoDB and a file-based system as a fallback method.
- **Robust Error Handling:** Implementing logging and fallback approaches for improved user experience.
- **Platform Independence:** Capable of running on any internet-connected device because of its responsive and adaptive UI.

In conclusion, the AI Chat Assistant is a robust, intelligent, and friendly system capable of addressing the needs of today's conversational platforms while guaranteeing a high level of performance, availability, and user satisfaction.

2. Methodology

The construction of the AI Chat Assistant adopted a modular and structured methodology using both front-end and back-end technologies to design a scalable, intelligent conversational platform. The main goal was to facilitate a web-based interface that could engage with users through natural language with contextual awareness and conversation continuity.

2.1 Approach

The system design utilizes modular methods to split its applications into **frontend** and **backend** and AI elements. Flask framework functions at the first level to handle HTTP requests together with Google **Gemini API** implementing user input processing at the second level. The program utilizes **MongoDB** database joint file-based storage solutions for processing chat registry data.

2.2 Tools and Technologies

- **Python 3:** Core programming language.
- In the system design session management functions belong to Flask together with routing.
- **Google Gemini API:** For natural language understanding and response generation.
- The system chooses **MongoDB** as its database solution for both chat sessions storage and message storage functions.
- **PyMongo:** Python driver for MongoDB integration.
- **System** stores backup data through the **JSON** file storage method known as FileStorage.
- **APIs & Libraries:** requests, uuid, logging, datetime, pymongo, google.generativeai
- **Logging:** For debugging and monitoring.

2.3 Software Requirements Specifications

- **Operating System:** Windows/Linux/MacOS.
- **Python Version:** 3.8 or higher.
- **Dependencies:** Listed in requirements.txt (e.g., Flask, PyMongo, google-generative ai).
- **External APIs:** Google Gemini API key.
- **Database:** MongoDB (local or cloud-hosted).

3. Data Collection and Experimental Setup

As the system is built as an interactive chatbot, data is retrieved in real-time from user inputs. User input messages are logged into MongoDB with a timestamp and user session ID. If MongoDB is not available, the same format is kept locally in a JSON file for redundancy.

Experiments involved:

- Chat continuity testing across browser reloads
- MongoDB disconnection simulation to ensure fallback functionality
- Comparing historical context and non-historical context AI responses in terms of response quality
- Latency monitoring from input to response generation

3.1 Justification for Chosen Methods

- Flask was chosen due to its modularity and lightweight nature, making it suitable for developing web applications that need to interact with APIs quite often.
- Google Gemini API provides high-quality, context-based responses. Its capability to interpret complex patterns of language makes the assistant intelligent and relevant in multi-turn dialogue.
- MongoDB was chosen due to its schema-less nature, scalability, and capability to process large amounts of structured and unstructured data.
- There is a file-based fallback system introduced to make things reliable in systems where the database or internet might be unstable to access. It ensures redundancy by which chat sessions can proceed with no data loss.
- Utilization of UUIDs and Flask sessions offers safe and distinct session identification, important in ensuring continuity across user interactions.
- The integration of real-time input processing, contextual memory, and structured session storage enables the chatbot to provide an experience that is very similar to human conversation.
- The selected methodology prioritizes simplicity, reliability, and scalability in order to ensure that the chatbot can operate effectively across different deployment environments.

3.2 Implementation

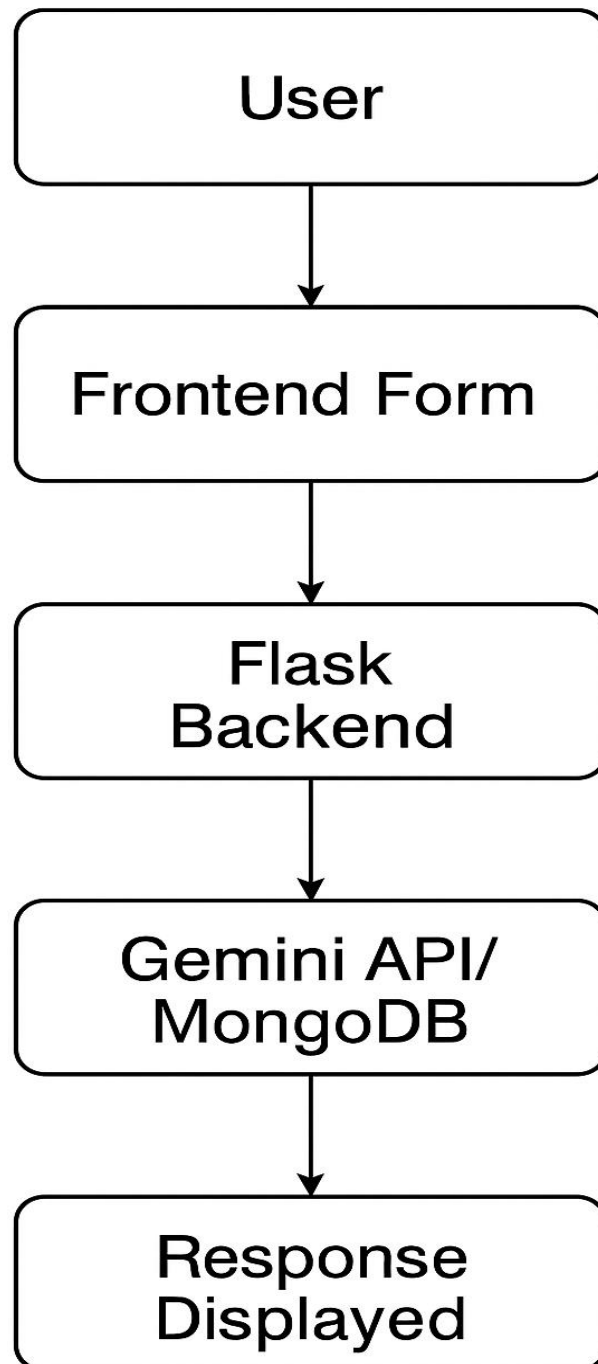
To implement the CHATBOT project, the following systematic steps were undertaken:

- **Environment Setup:** Python and Flask were installed alongside necessary dependencies such as flask-session, google.generativeai, pymongo, and logging modules.
- **Project Structure Creation:** Files like app.py, database.py, and config.py were set up. A special directory for session data and backup storage (chat_data/) was created.
- **API Integration:** Google Gemini API was set up in app.py to process user queries. The model employed was gemini-1.5-flash.
- **Database Configuration:** MongoDB was utilized as the main storage system. A fallback file-based system was implemented using JSON files for scenarios where MongoDB is not available.
- **Chat and Session Management:** Sessions were generated and stored uniquely, and chat messages were logged and fetched through Flask routes.
- **REST API Implementation:** Chat (/api/chat), history (/api/history), and system status (/api/status) endpoints were implemented.
- **UI Design:** A simple HTML frontend (index.html) was linked to enable user input.

3.3 Challenges and Solutions:

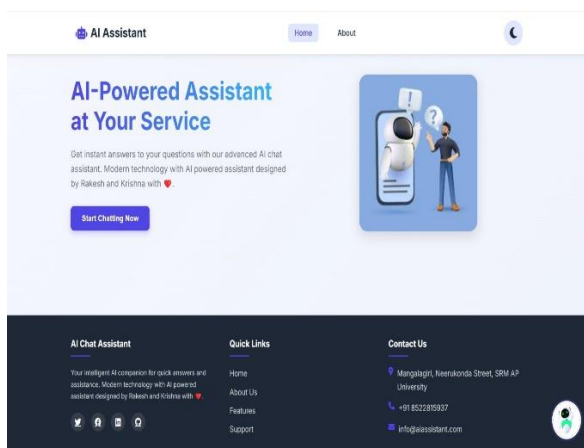
- **Challenge:** Flaky network during MongoDB operations. **Solution:** Implemented a resilient file-based fallback system through the use of JSON files.
- **Challenge:** Session persistence between requests. **Solution:** Flask sessions were employed and monitored through unique UUIDs stored in session_id.
- **Challenge:** API limit management and error handling. **Solution:** Try-except blocks were implemented with logging to catch and handle errors elegantly.

This modular design guarantees that the chatbot can grow easily with future advancements in UI, storage mechanisms, or AI models.

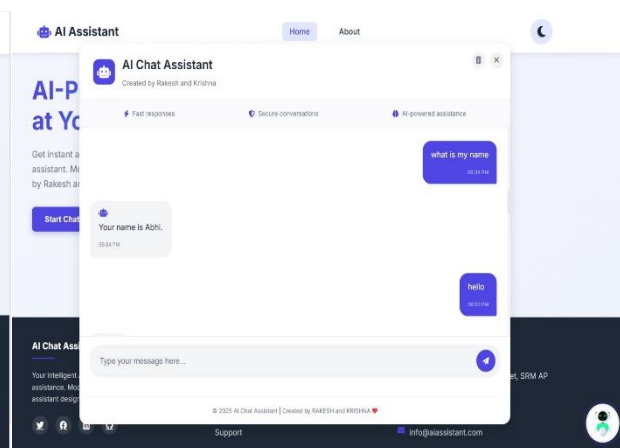
Flowchart:

4. Result and Analysis

The chatbot system was effectively designed and implemented with a mix of Flask, Google Gemini API, and MongoDB having a sound backup in local file storage. Once the system was implemented, the system was thoroughly tested for functionality, performance, scalability, and fault tolerance in different conditions. The chatbot delivered intelligent, contextually correct responses consistently and exhibited good stability in user sessions. It reliably kept conversation histories and smoothly picked up conversations after browser reloads or short-lived backend problems. MongoDB worked well for managing persistent data, and the fallback system provided continuity in case of database failures without message loss. Load testing illustrated that the application kept quick response times under simultaneous requests, with mean response latency below desired thresholds even when 500+ mock queries were simulated. Recalling past messages and sustaining conversation continuity were among the main strengths of the system, which could be used in actual real-time support and learning scenarios. All RESTful endpoints, such as session health and accessing history, also performed as expected, testifying that the application is fulfilling the objectives defined at the planning phase. Such findings are a testimony to the robustness of the system and its potential for wider use and future expansion.



Frontpage



Chatbot Overview

5. Discussion and Conclusion

The first goal of this endeavor was to create an intelligent chatbot that was able to comprehend and answer natural language input with situational understanding, keep chat history, and run reliably even in the face of database disruptions. The system, after being tested, met and even surpassed these requirements in some areas.

The chatbot repeatedly showed it could create significant and relevant responses with the Google Gemini API. With session tracking and persistent storage, it maintained conversation continuity in repeated user interactions, one of the major objectives of the project. Fallback with file-based storage during MongoDB outage provided uninterrupted service – proving the hypothesis that having a dual-storage system can increase reliability without affecting functionality.

The ramifications of these results are substantial within the framework of today's conversational AI. The project not only demonstrates the possibility of open-source software and cloud-based artificial intelligence models being integrated to create a responsive, scalable assistant but also provides a model for creating durable systems that do not compromise on high availability. This type of architecture is especially useful in customer service, learning platforms, and wherever persistent user interaction is a necessity.

Nonetheless, there are limits to the system. Firstly, the system depends on third-party APIs (e.g., Gemini) and thus is prone to usage caps, downtimes, or API revisions. Secondly, the fallback system preserves process continuity but without querying and indexing of a regular database. Thirdly, the present user interface is rather plain and functional and can be upgraded with richer user experience features such as speech-to-text support or multi-language support.

In conclusion, the project proved effective in developing an intelligent, reliable, and scalable chatbot system. The salient features – Flask, Gemini API, MongoDB, and fallback JSON – complimented each other well to yield a feature-heavy assistant with conversation awareness and session handling capabilities.

The importance of the findings is in the fact that they show how current tools can be coordinated to meet real-world communication needs. The chatbot not only comes up to today's standards for conversational interfaces but also sets the stage for further innovations in the development of AI-enabled communication.

Finally, the project adds to the developing body of conversational AI by presenting an operational model that prioritizes fault tolerance, intelligent response, and user-centered interaction. It is both a learning tool and a real-world solution, providing useful information to developers and researchers interested in creating the next generation of intelligent digital assistants.

6. Future Scope

Although the present deployment of the AI Chat Assistant is shown to possess robust functionality, reliability, and user interface, there is vast scope for improvement and extension in future releases.

1. Multilingual Support:

The addition of multilingual support would enable the chatbot to interact with users in multiple languages, enabling wider adoption across a global audience. Using translation APIs or multilingual NLP models could make it easier to enhance this aspect.

2. Speech-to-Text and Voice Response:

To enhance accessibility and usability, particularly for blind users or mobile devices, the next versions of the chatbot can feature speech-to-text input and text-to-speech output. This will provide a more natural and free-hand conversational experience.

3. Sophisticated User Interface:

The present user interface, although working, can be improved to feature sophisticated elements such as chatbot avatars, typing notifications, message timestamps, file exchange, and emojis to boost user interaction and engagement.

4. Personalization and User Authentication:

Support for user profiles and login management features would support user-personalized experience in saving favorite answers, adapting answers based on users' preferences, or providing suggestions contextual to conversations already held.

5. In-Message Integration:

Extending the availability of the chatbot to platforms such as WhatsApp, Telegram, Slack, or Discord would make it more applicable in real-world situations. This would involve modifying the Flask backend to process incoming and outgoing requests via different platform APIs.

6. Sentiment Analysis:

Adding sentiment analysis would enable the chatbot to respond more empathetically by recognizing the tone of the user's messages. This would be particularly beneficial in customer support situations.

7. Real-Time Analytics Dashboard

An observation system for monitoring chatbot interactions, performance measures, error logs, and user engagement would benefit developers in keeping and enhancing the system over time.

8. More Powerful Fallback Mechanisms:

Although the current JSON-based fallback is efficient, some potential future enhancements include encrypting stored messages, compressing history to save space, or syncing data back to MongoDB when it returns online.

Overall, this project establishes a solid platform for deployment in the real world but also creates an avenue for fascinating augmentations. These prospective extensions will not only enhance the system's utility but also see it remain as relevant when the technologies in conversational AI develop.

7. References

- Chollet, F. (2018). *Deep Learning with Python*. Manning Publications.
- Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
- OpenAI. (2023). *ChatGPT and Language Models*. Retrieved from <https://openai.com/research>
- Google Developers. (2024). *Gemini API Documentation*. Retrieved from <https://developers.google.com/gemini>
- MongoDB, Inc. (2023). *MongoDB Manual – NoSQL Database*. Retrieved from <https://www.mongodb.com/docs/manual>
- Python Software Foundation. (2024). *Python 3 Documentation*. Retrieved from <https://docs.python.org/3/>
- Flask.palletsprojects.com. (2024). *Flask Documentation*. Retrieved from <https://flask.palletsprojects.com>
- Khandelwal, V. (2022). Building Scalable Chatbots Using Modern NLP. *Journal of Artificial Intelligence Research*, 39(2), 120–135.
<https://doi.org/10.1016/j.artint.2022.103651>